

Computer Vision – Lecture 2

Image Processing I

10.04.2025

Bastian Leibe

Visual Computing Institute

RWTH Aachen University

<http://www.vision.rwth-aachen.de/>

leibe@vision.rwth-aachen.de

Course Outline

- Image Processing Basics
 - Image Formation
 - Linear Filters
 - Gradient Filters and Edge Detection
 - Structure Extraction
- Segmentation
- Object Recognition and Categorization
- Deep Learning Approaches for Vision
- 3D Reconstruction

“Flipped Classroom” Model

Lecture 2: Image Processing I (11.04.2022, online)

This lecture slot will be given online. Please use the following permanent zoom link to attend:

<https://rwth.zoom.us/j/98615042972?pwd=enNvUjZlZjNNQ2VkMW9zV3E2TC9vUT09>

Passcode: 914166

 cv22-part02-image-processing1.pdf
2.2MB Hochgeladen 8.04.2022 23:11

 cv22-part02-image-processing1-6on1.pdf
1.5MB Hochgeladen 8.04.2022 23:12

 Pre-recorded videos

"Flipped classroom" model: Please watch the following pre-recorded videos before the lecture on Monday 11.04. The lecture slot will then contain a Q&A on the topics covered in the videos.

 cv22-part02a-image-processing1-qa.pdf
1.3MB Hochgeladen 8.04.2022 23:44

Pre-recorded videos

Part 1.1: Image Formation

Computer Vision – Lecture 1

Introduction

1.2 Image Formation

Bastian Leibe

Visual Computing Institute
RWTH Aachen University
<http://www.vision.rwth-aachen.de/>

- Pre-recorded videos made available on moodle
 - Part 1.1: Image Formation
 - Part 2.1: Linear Filters
 - Part 2.2: Background on Linear Filters
- *You can use those to rehearse the content of this lecture*

Reminder: Flipped Classroom Format

- “Flipped Classroom” Model
 - Pre-recorded videos of the lecture contents are made available before the lecture slots
 - You are expected to watch and rehearse the lecture contents ahead of the lecture (!)
 - The lecture itself will then take the form of a Q&A session
 - Short repetition + opportunity to ask questions

Motivation

- Noise reduction/image restoration



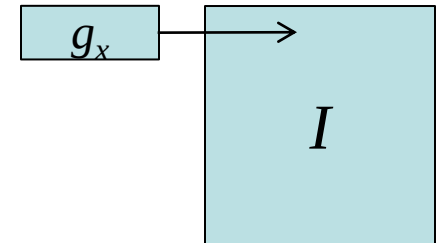
- Structure extraction



Topics of This Lecture

- Linear filters

- What are they? How are they applied?
- Application: Image Smoothing
- Gaussian filter
- What does it *mean* to filter an image?



- Nonlinear Filters

- Median filter

- Multi-Scale representations

- How to properly rescale an image?

- Filters as templates

- Correlation as template matching

Common Types of Noise

- Salt & pepper noise
 - Random occurrences of black and white pixels
- Impulse noise
 - Random occurrences of white pixels
- Gaussian noise
 - Variations in intensity drawn from a Gaussian (“Normal”) distribution.
- *Basic Assumption*
 - *Noise is i.i.d. (independent & identically distributed)*



Original



Salt and pepper noise

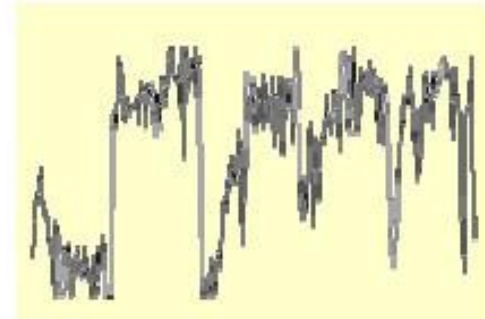
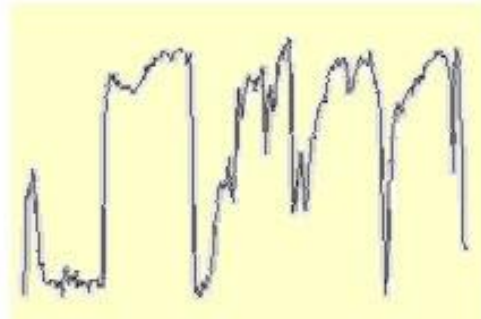
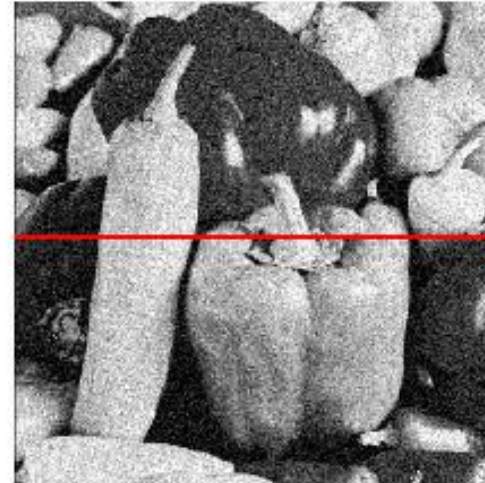


Impulse noise



Gaussian noise

Gaussian Noise



$$f(x, y) = \underbrace{\hat{f}(x, y)}_{\text{Ideal Image}} + \underbrace{\eta(x, y)}_{\text{Noise process}}$$

Gaussian i.i.d. ("white") noise:
 $\eta(x, y) \sim \mathcal{N}(\mu, \sigma)$

```
>> noise = randn(size(im)).*sigma;
```

```
>> output = im + noise;
```


First Attempt at a Solution

- Assumptions:
 - Expect pixels to be like their neighbors
 - Expect noise processes to be independent from pixel to pixel (“i.i.d. = independent, identically distributed”)
- Let’s try to replace each pixel with an average of all the values in its neighborhood...

Moving Average in 2D

$$F[x, y]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$G[x, y]$$

	0								

Moving Average in 2D

$$F[x, y]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$G[x, y]$$

	0	10							

Moving Average in 2D

$$F[x, y]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$G[x, y]$$

	0	10	20						

Moving Average in 2D

$$F[x, y]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$G[x, y]$$

	0	10	20	30					

Moving Average in 2D

$$F[x, y]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$G[x, y]$$

	0	10	20	30	30				

Moving Average in 2D

$$F[x, y]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$$G[x, y]$$

	0	10	20	30	30	30	20	10	
	0	20	40	60	60	60	40	20	
	0	30	60	90	90	90	60	30	
	0	30	50	80	80	90	60	30	
	0	30	50	80	80	90	60	30	
	0	20	30	50	50	60	40	20	
	10	20	30	30	30	30	20	10	
	10	10	10	0	0	0	0	0	

Correlation Filtering

- Say the averaging window size is $2k+1 \times 2k+1$:

$$G[i, j] = \underbrace{\frac{1}{(2k+1)^2}}_{\text{Attribute uniform weight to each pixel}} \underbrace{\sum_{u=-k}^k \sum_{v=-k}^k F[i+u, j+v]}_{\text{Loop over all pixels in neighborhood around image pixel } F[i,j]}$$

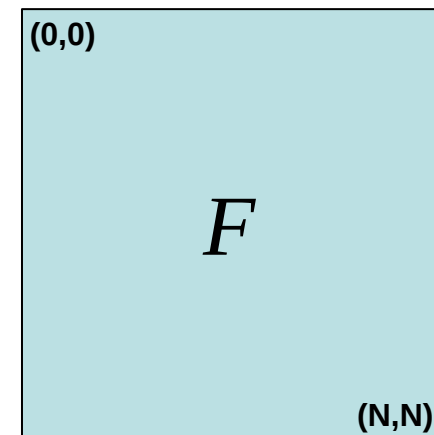
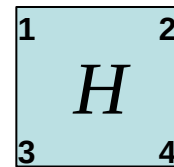
- Now generalize to allow different weights depending on neighboring pixel's relative position:

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k \underbrace{H[u, v] F[i+u, j+v]}_{\text{Non-uniform weights}}$$

Correlation Filtering

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

- This is called **cross-correlation**, denoted $G = H \otimes F$
- Filtering an image
 - Replace each pixel by a weighted combination of its neighbors.
 - The filter “kernel” or “mask” is the prescription for the weights in the linear combination.



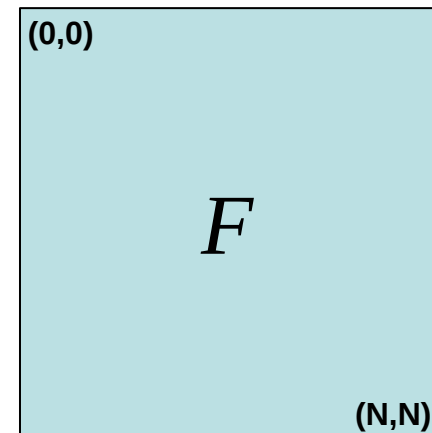
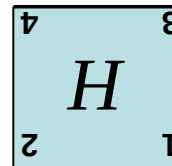
Convolution

- Convolution:
 - Flip the filter in both dimensions (bottom to top, right to left)
 - Then apply cross-correlation

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i - u, j - v]$$

$$G = H \star F$$

↑
*Notation for
convolution
operator*



Correlation vs. Convolution

- Correlation

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

$$G = H \otimes F$$

- Convolution

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i - u, j - v]$$

$$G = H \star F$$

- Note

- If $H[-u, -v] = H[u, v]$, then correlation $\equiv$ convolution.

Note the difference!



Shift Invariant Linear System

- Shift invariant:
 - Operator behaves the same everywhere, *i.e.* the value of the output depends on the pattern in the image neighborhood, not the position of the neighborhood.
- Linear:
 - Superposition: $h \star (f_1 + f_2) = (h \star f_1) + (h \star f_2)$
 - Scaling: $h \star (kf) = k(h \star f)$

Properties of Convolution

- Linear & shift invariant
- Commutative: $f \star g = g \star f$
- Associative: $(f \star g) \star h = f \star (g \star h)$
 - Often apply several filters in sequence: $((a \star b_1) \star b_2) \star b_3$
 - This is equivalent to applying one filter: $a \star (b_1 \star b_2 \star b_3)$
- Identity: $f \star e = f$
 - for unit impulse $e = [\dots, 0, 0, 1, 0, 0, \dots]$.
- Differentiation: $\frac{\partial}{\partial x}(f \star g) = \frac{\partial f}{\partial x} \star g$

Averaging Filter

- What values belong in the kernel $H[u, v]$ for the moving average example?

$$F[x, y] \otimes H[u, v] = G[x, y]$$

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	90	0	90	90	90	0	0
0	0	0	90	90	90	90	90	0	0
0	0	0	0	0	0	0	0	0	0
0	0	90	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

$\otimes$
 $\frac{1}{9}$

1	1	1
1	?	1
1	1	1

$=$
 $G[x, y]$

	0	10	20	30	30				

“box filter”

$$G = H \otimes F$$

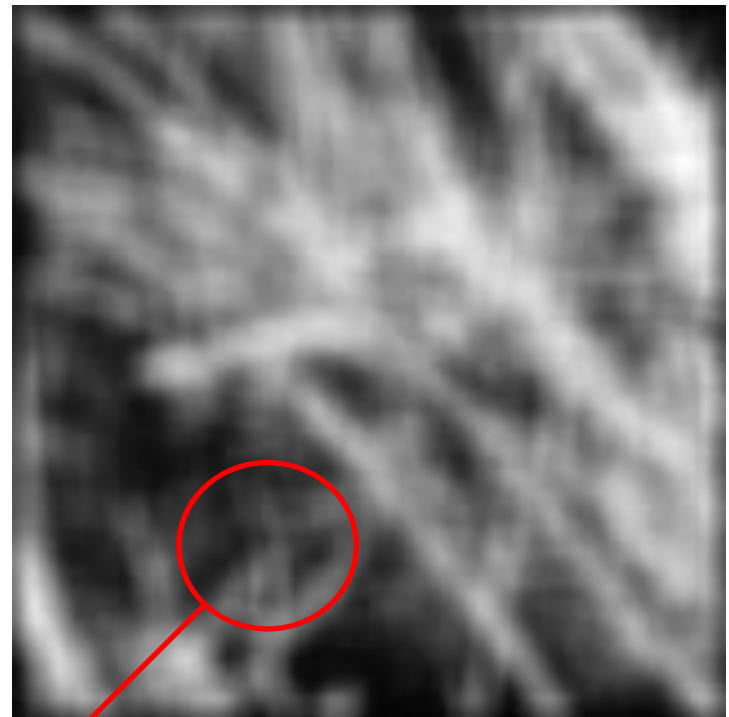
Smoothing by Averaging



depicts box filter:
white = high value, black = low value



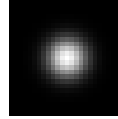
Original



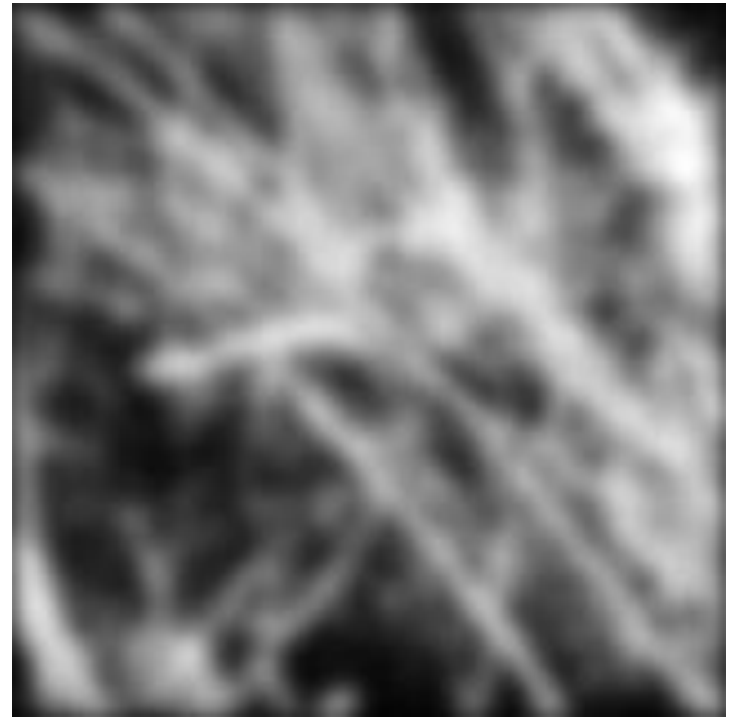
Filtered

“Ringing” artifacts!

Smoothing with a Gaussian

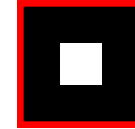
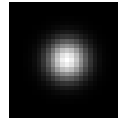


Original



Filtered

Smoothing with a Gaussian – Comparison



Original



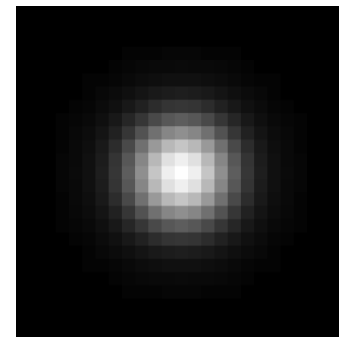
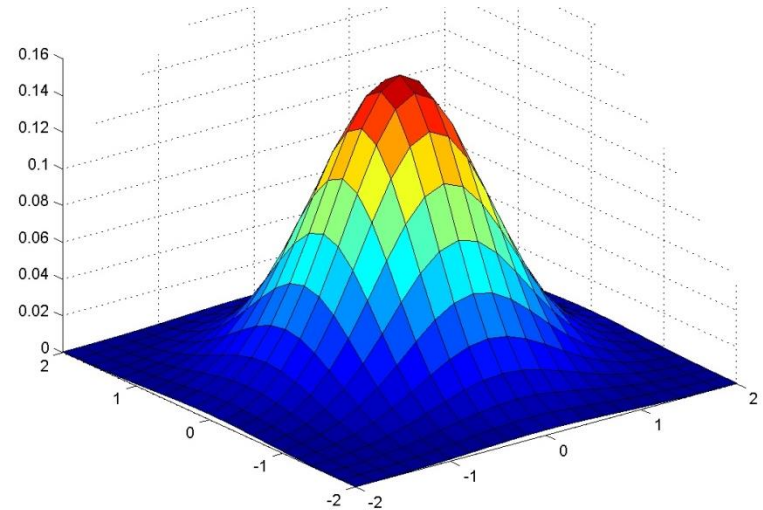
Filtered

Gaussian Smoothing

- Gaussian kernel

$$G_{\sigma} = \frac{1}{2\pi\sigma^2} e^{-\frac{(x^2+y^2)}{2\sigma^2}}$$

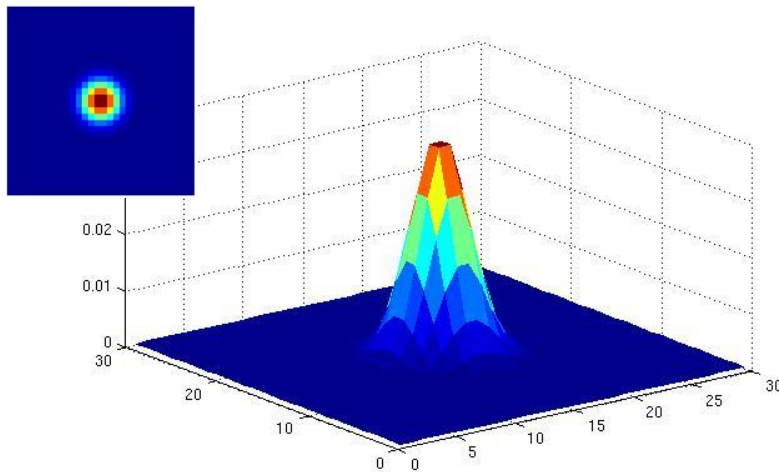
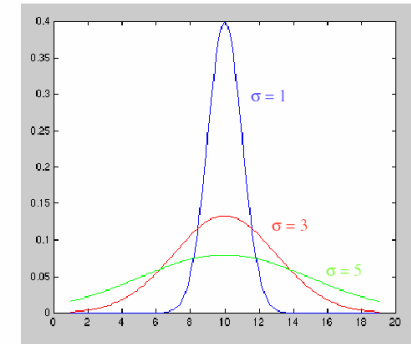
- Rotationally symmetric
- Weights nearby pixels more than distant ones
 - This makes sense as “probabilistic” inference about the signal
- A Gaussian gives a good model of a fuzzy blob



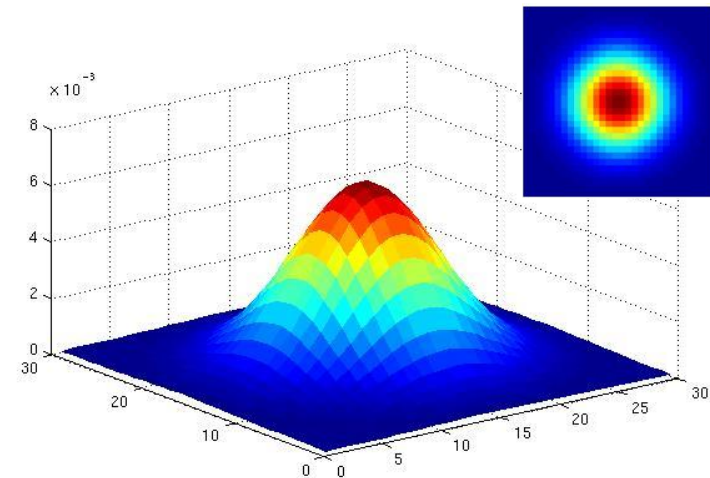
Gaussian Smoothing

- What parameters matter here?
- *Variance* σ^2 of Gaussian
 - Determines extent of smoothing

Effect of σ



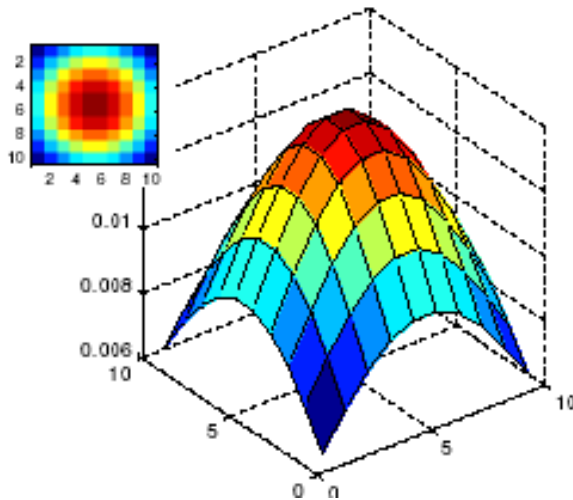
$\sigma = 2$ with 30×30
kernel



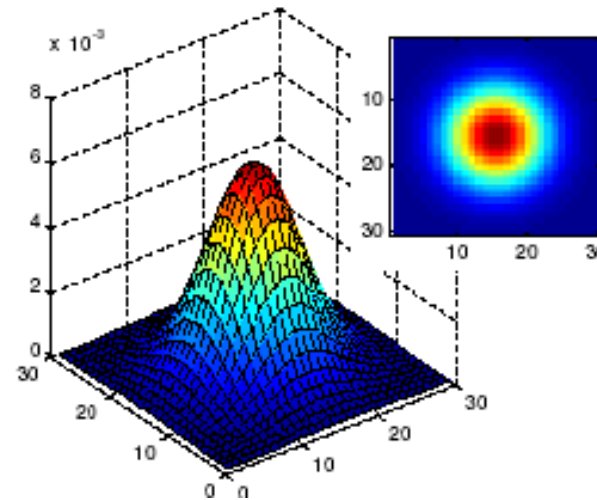
$\sigma = 5$ with 30×30
kernel

Gaussian Smoothing

- What parameters matter here?
- *Size* of kernel or mask
 - Gaussian function has infinite support, but discrete filters use finite kernels



$\sigma = 5$ with 10×10
kernel



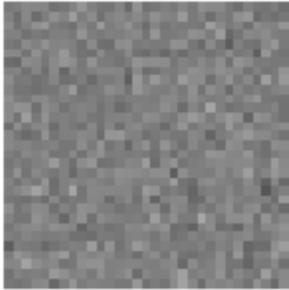
$\sigma = 5$ with 30×30
kernel

- Rule of thumb: set filter half-width to about 3σ !

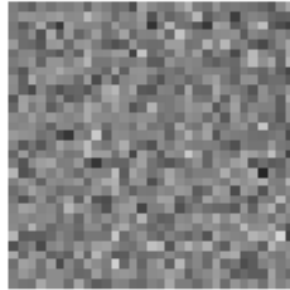
Effect of Smoothing

More noise →

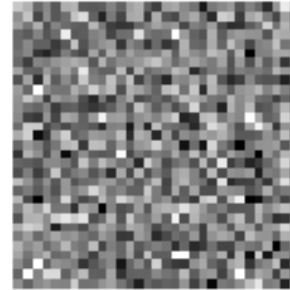
$\sigma=0.05$



$\sigma=0.1$



$\sigma=0.2$



no
smoothing

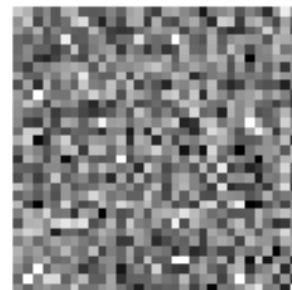
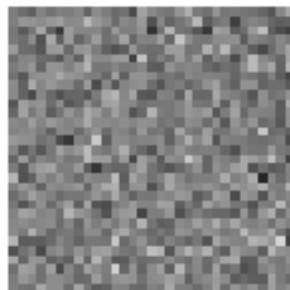
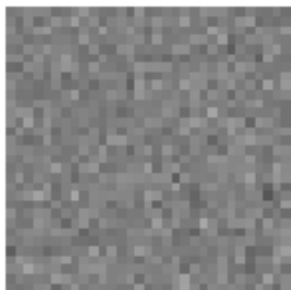
Effect of Smoothing

More noise →

$\sigma=0.05$

$\sigma=0.1$

$\sigma=0.2$



no
smoothing



$\sigma=1$ pixel



$\sigma=2$ pixels

Wider smoothing kernel →



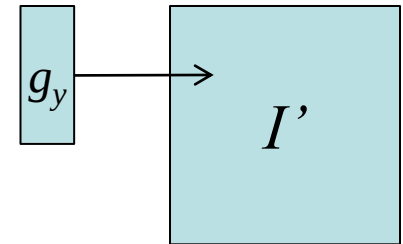
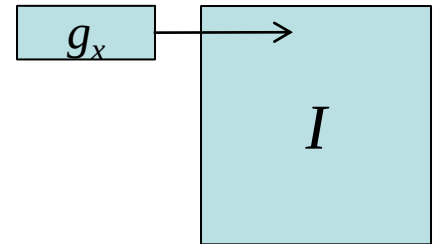
Efficient Implementation

- Both, the BOX filter and the Gaussian filter are separable:
 - First convolve each row with a 1D filter

$$g(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp(-x^2 / (2\sigma^2))$$

- Then convolve each column with a 1D filter

$$g(y) = \frac{1}{\sqrt{2\pi}\sigma} \exp(-y^2 / (2\sigma^2))$$

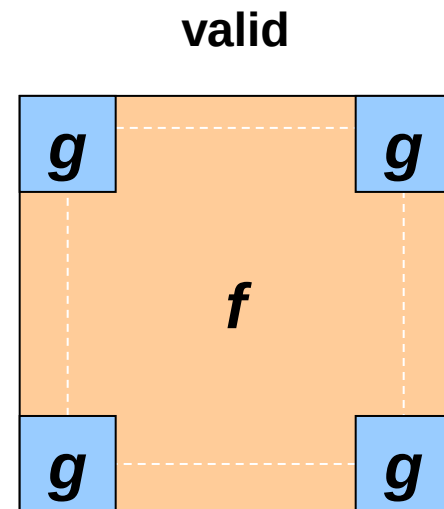
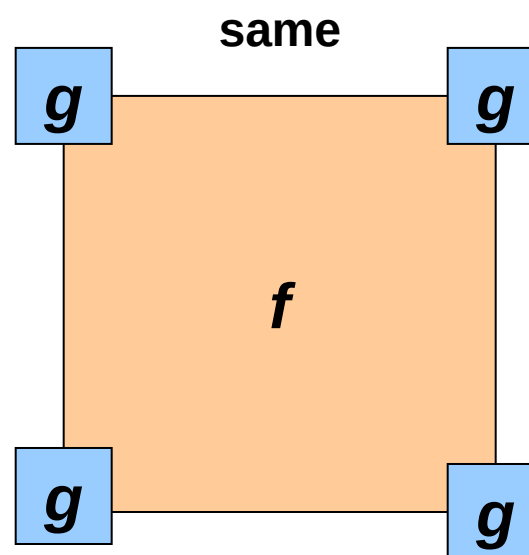
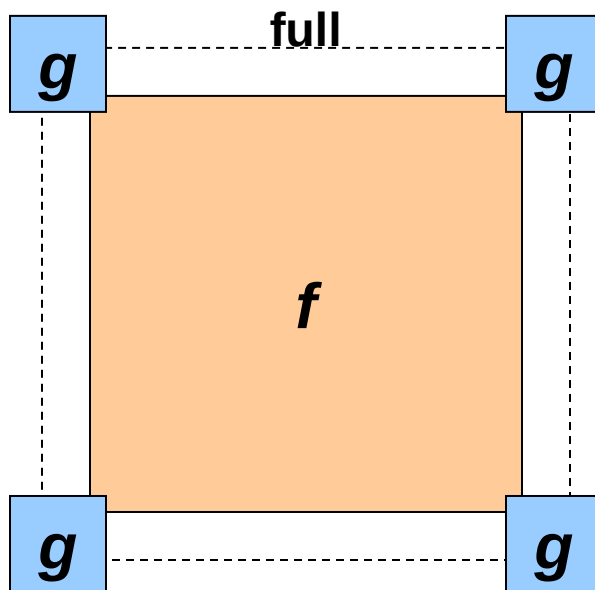


- Remember:
 - Convolution is linear – associative and commutative

$$g_x \star g_y \star I = g_x \star (g_y \star I) = (g_x \star g_y) \star I$$

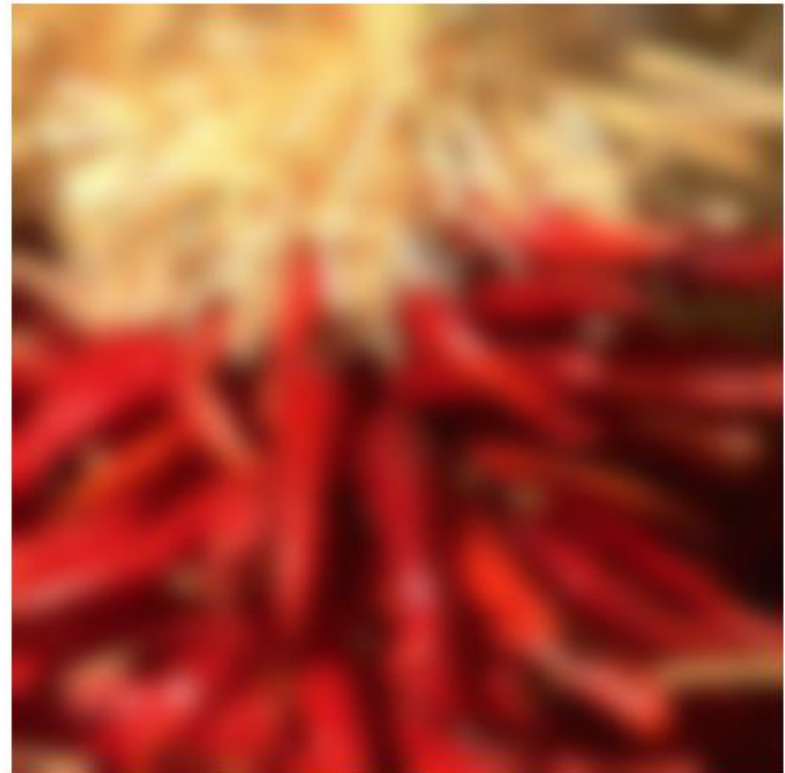
Filtering: Boundary Issues

- What is the size of the output?
- MATLAB: `filter2(g, f, shape)`
 - `shape` = 'full': output size is sum of sizes of `f` and `g`
 - `shape` = 'same': output size is same as `f`
 - `shape` = 'valid': output size is difference of sizes of `f` and `g`



Filtering: Boundary Issues

- How should the filter behave near the image boundary?
 - The filter window falls off the edge of the image
 - Need to extrapolate
 - Methods:
 - Clip filter (black)
 - Wrap around
 - Copy edge
 - Reflect across edge



Filtering: Boundary Issues

- How should the filter behave near the image boundary?
 - The filter window falls off the edge of the image
 - Need to extrapolate
 - Methods (MATLAB):
 - Clip filter (black): `imfilter(f,g,0)`
 - Wrap around: `imfilter(f,g,'circular')`
 - Copy edge: `imfilter(f,g,'replicate')`
 - Reflect across edge: `imfilter(f,g,'symmetric')`

Topics of This Lecture

- Linear filters
 - What are they? How are they applied?
 - Application: smoothing
 - Gaussian filter
 - What does it *mean* to filter an image?
- Nonlinear Filters
 - Median filter
- Multi-Scale representations
 - How to properly rescale an image?
- Filters as templates
 - Correlation as template matching

Why Does This Work?

- Let's take a small excursion into the Fourier domain to talk about spatial frequencies...



$$3 \cos(x)$$

A

$$+ 1 \cos(3x)$$

B

$$+ 0.8 \cos(5x)$$

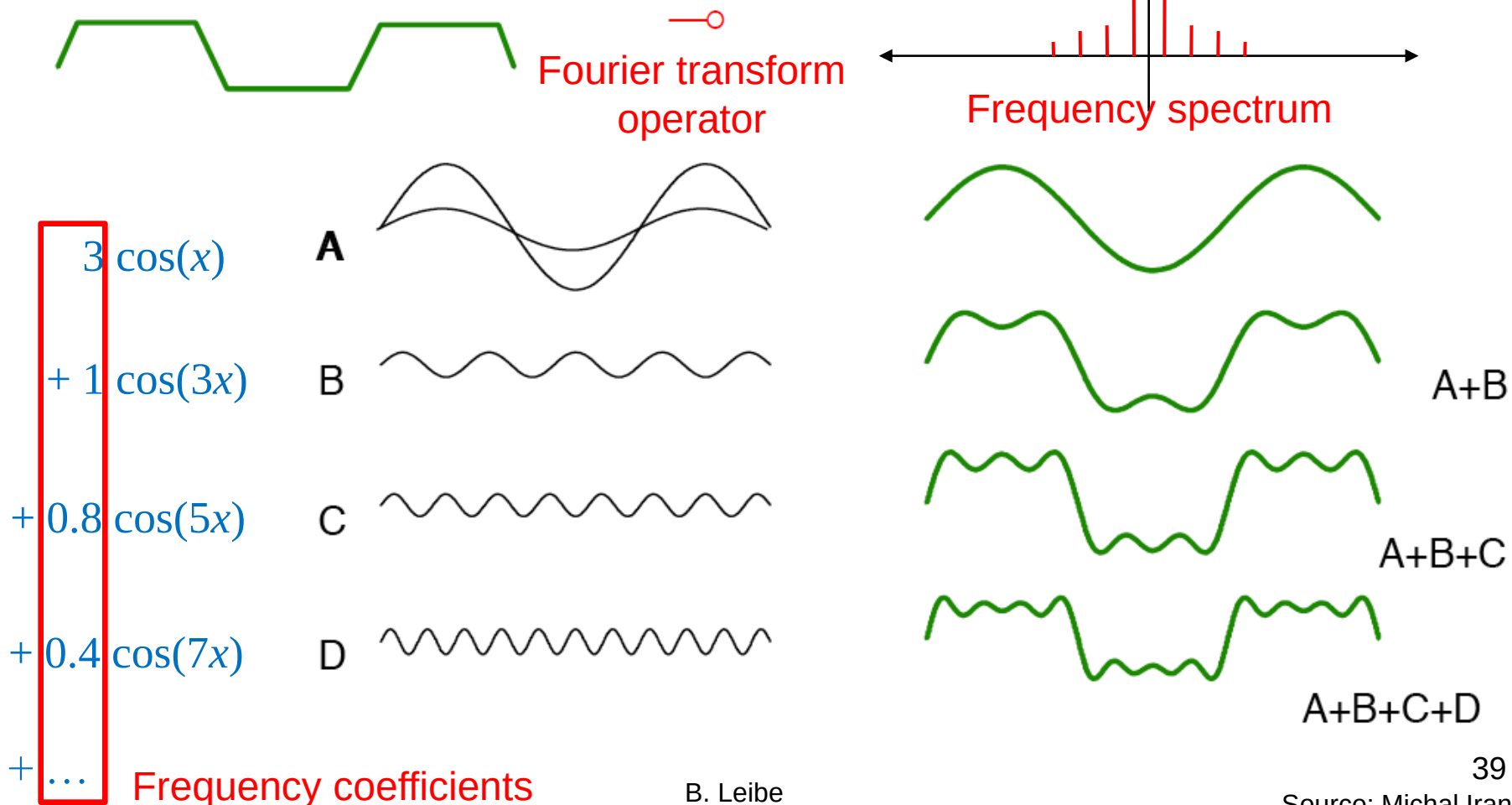
C

$$+ 0.4 \cos(7x)$$

D $+ \dots$ **A+B****A+B+C****A+B+C+D**

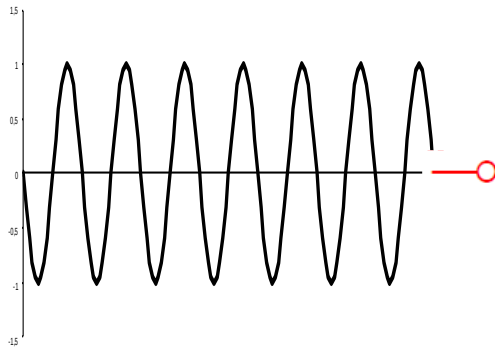
The Fourier Transform in Cartoons

- A small excursion into the Fourier transform to talk about spatial frequencies...

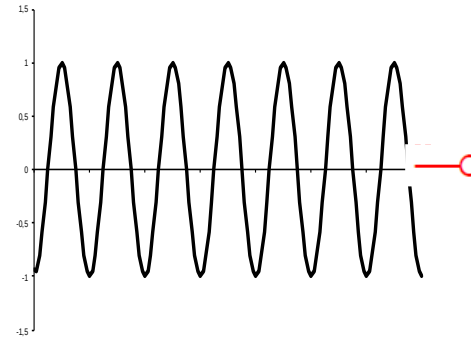


Fourier Transforms of Important Functions

- Sine and cosine transform to...



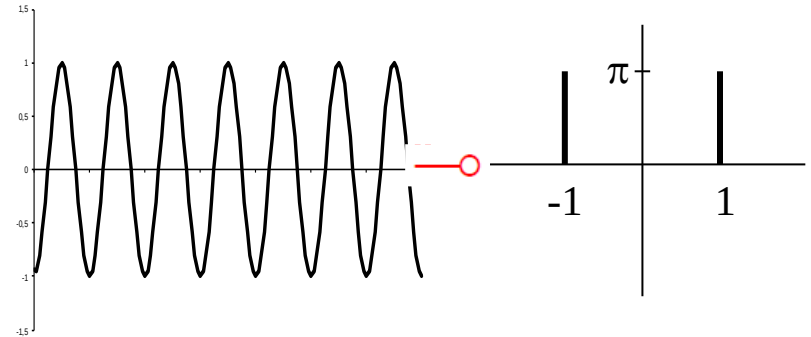
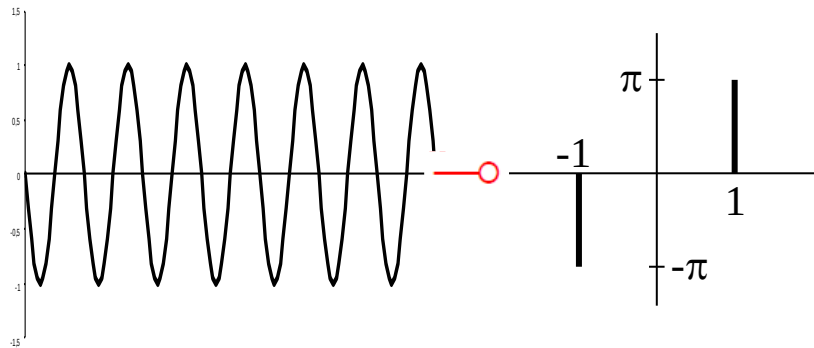
?



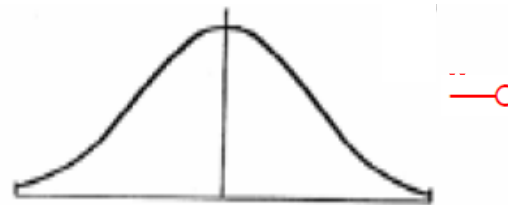
?

Fourier Transforms of Important Functions

- Sine and cosine transform to “frequency spikes”



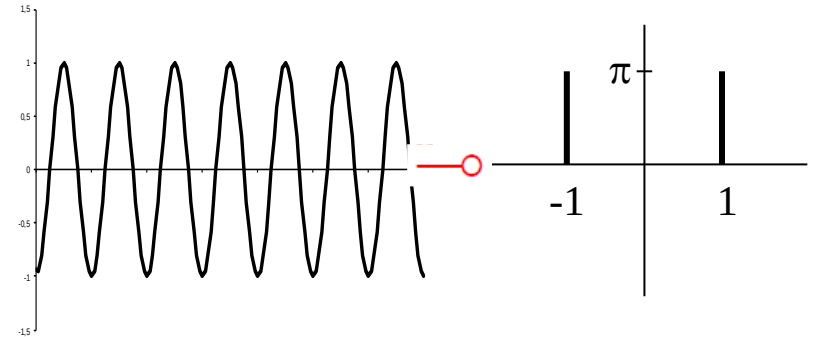
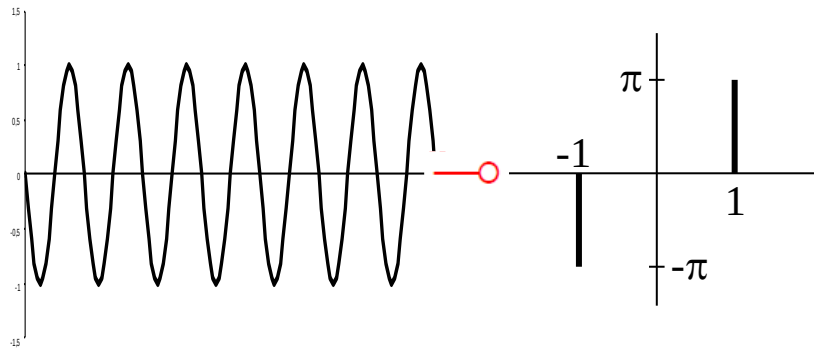
- A Gaussian transforms to...



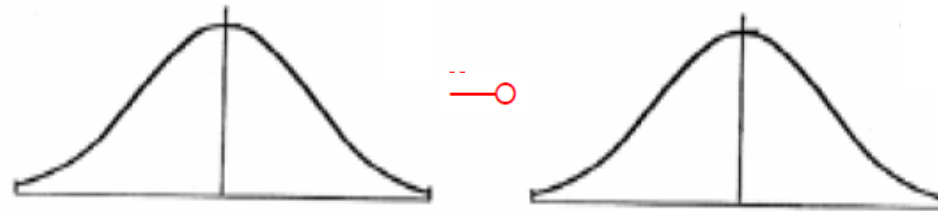
?

Fourier Transforms of Important Functions

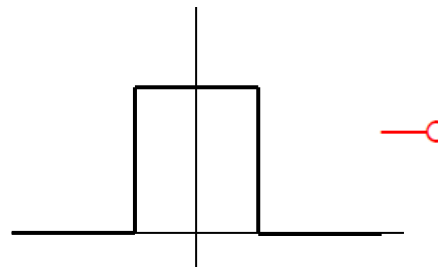
- Sine and cosine transform to “frequency spikes”



- A Gaussian transforms to a Gaussian



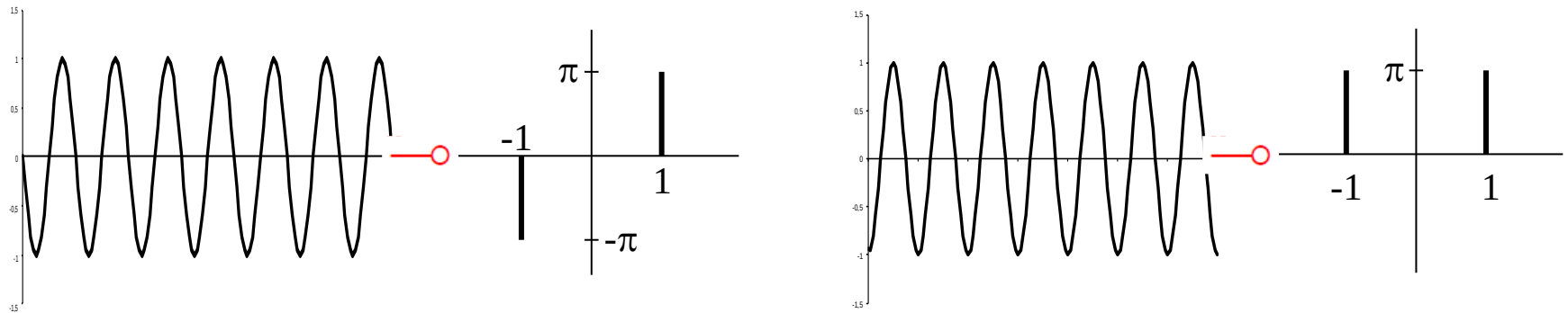
- A box filter transforms to...



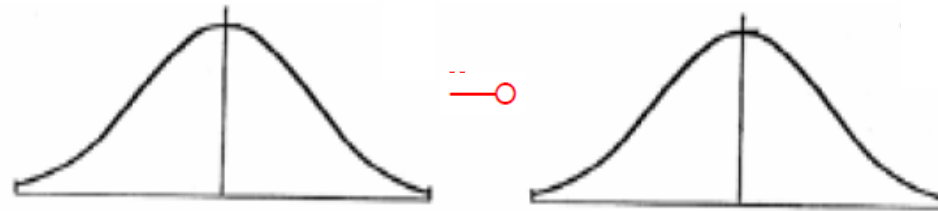
?

Fourier Transforms of Important Functions

- Sine and cosine transform to “frequency spikes”

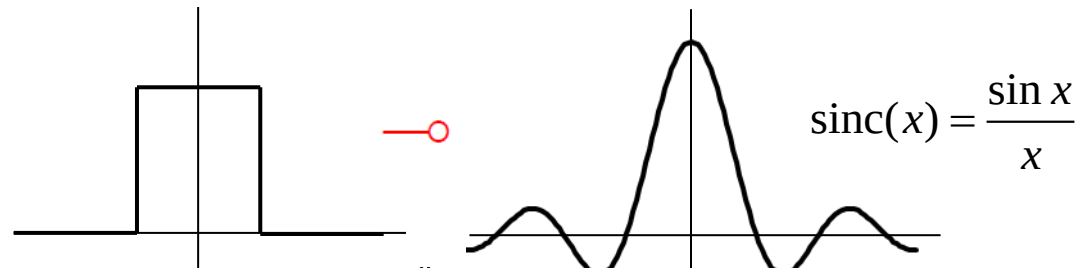


- A Gaussian transforms to a Gaussian



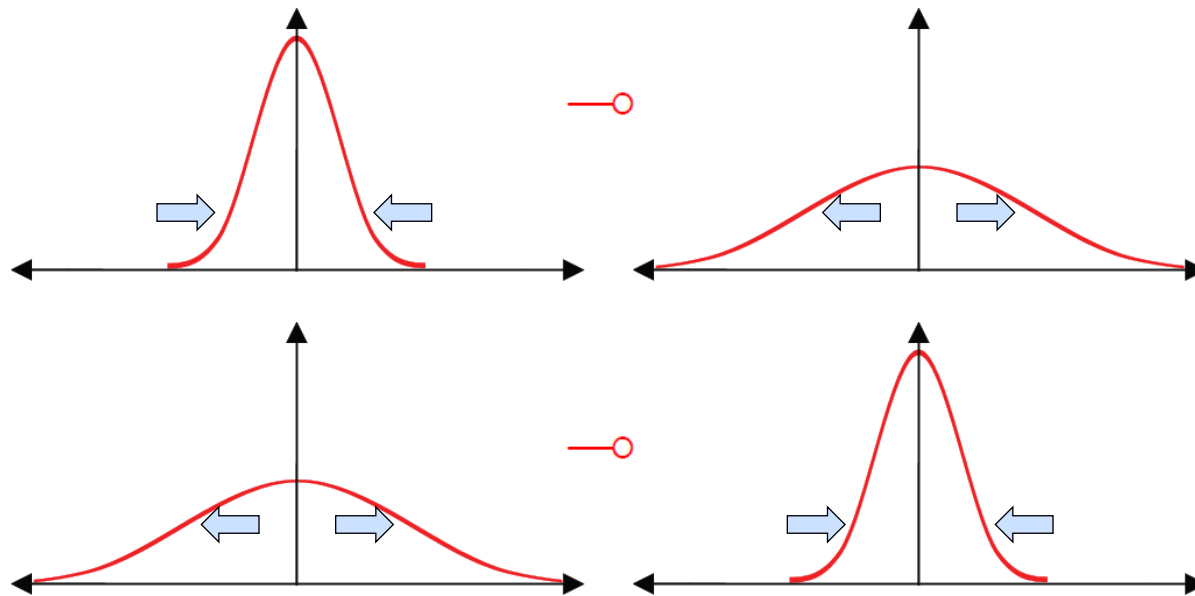
All of this is symmetric!

- A box filter transforms to a sinc

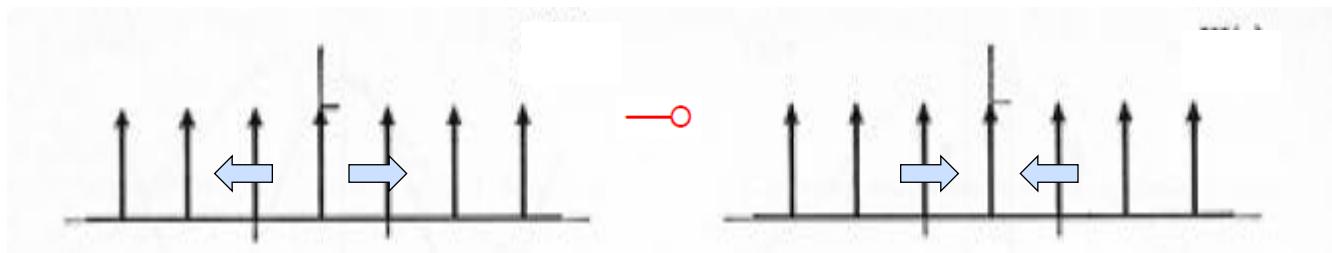


Duality

- The better a function is localized in one domain, the worse it is localized in the other.



- This is true for any function



Effect of Convolution

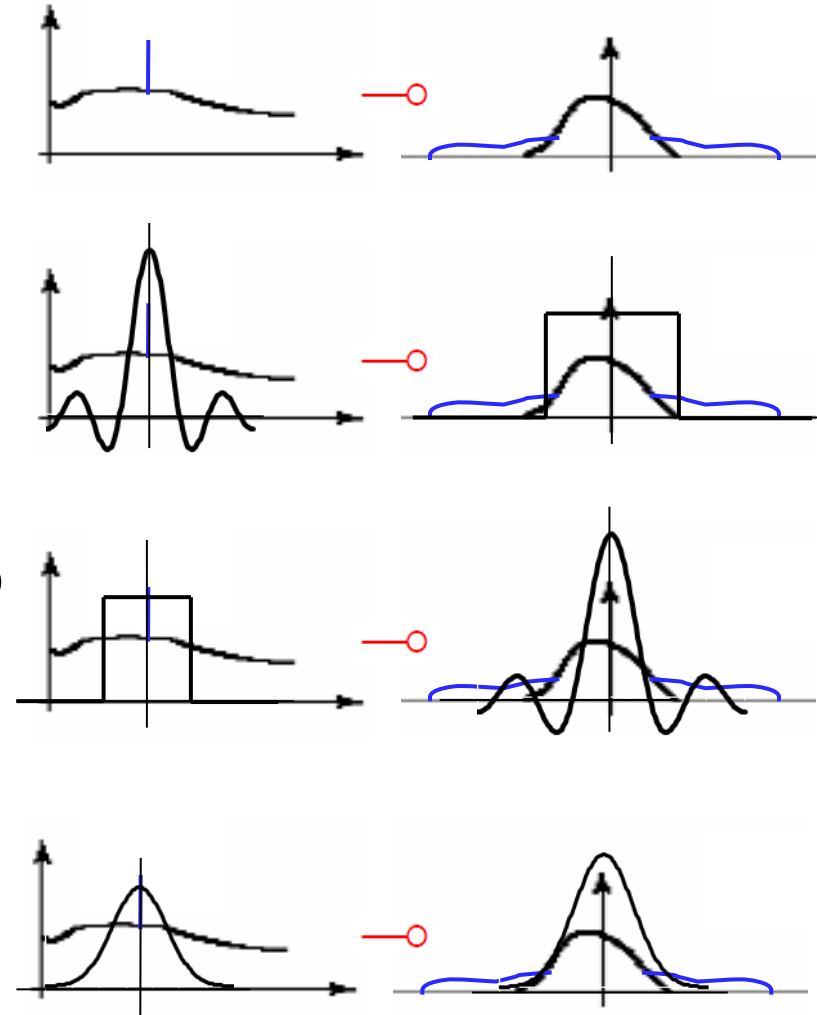
- Convoluting two functions in the image domain corresponds to taking the product of their transformed versions in the frequency domain.

$$f \star g \longrightarrow \mathcal{F} \cdot \mathcal{G}$$

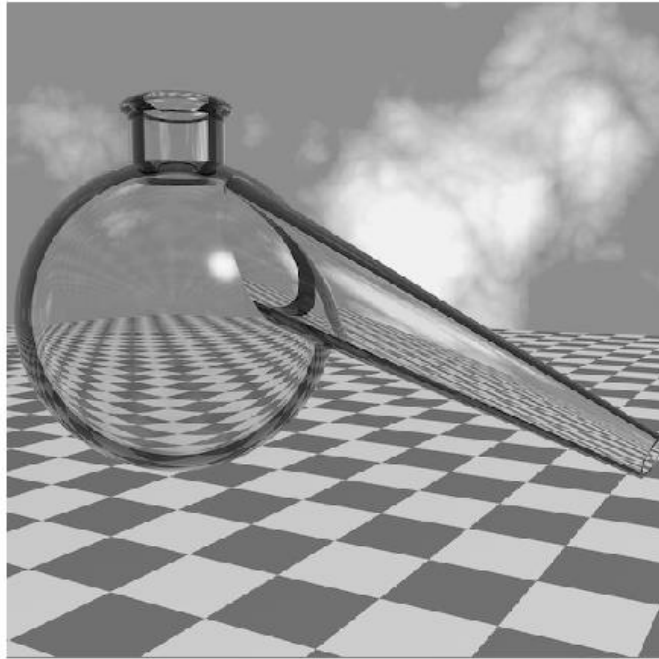
- This gives us a tool to manipulate image spectra.
 - A filter attenuates or enhances certain frequencies through this effect.

Effect of Filtering

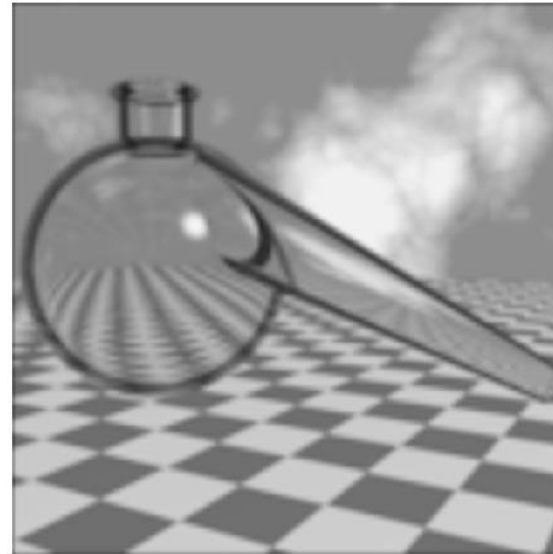
- Noise introduces high frequencies. To remove them, we want to apply a “low-pass” filter.
- The ideal filter shape in the frequency domain would be a box. But this transfers to a spatial sinc, which has infinite spatial support.
- A compact spatial box filter transfers to a frequency sinc, which creates artifacts.
- A Gaussian has compact support in both domains. This makes it a convenient choice for a low-pass filter.



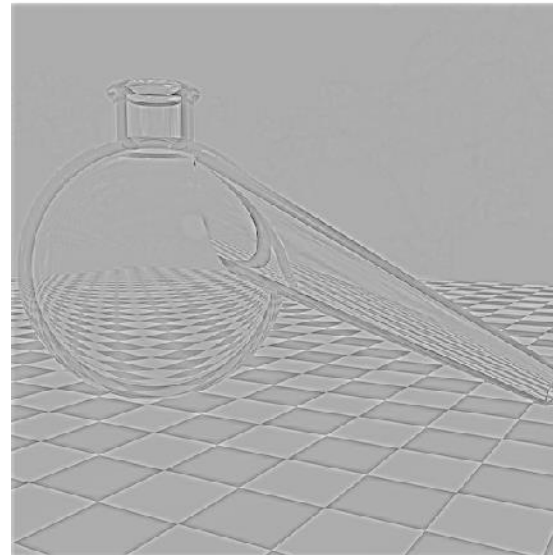
Low-Pass vs. High-Pass



Original image

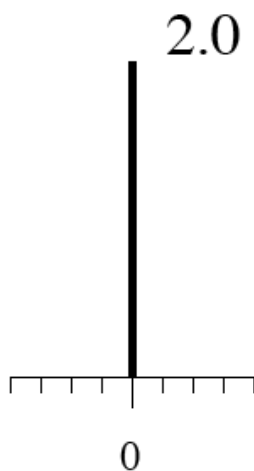


Low-pass
filtered

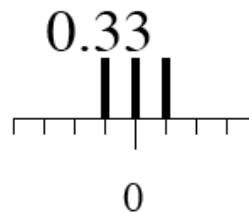


High-pass
filtered

Quiz: What Effect Does This Filter Have?



—

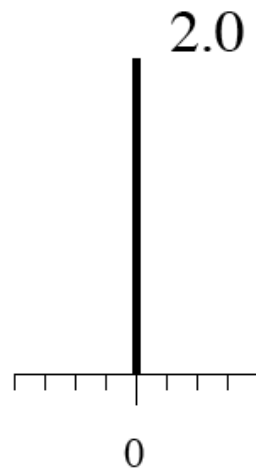


?

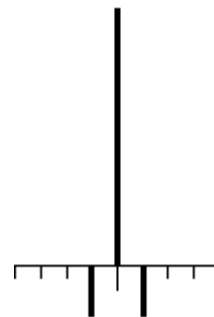
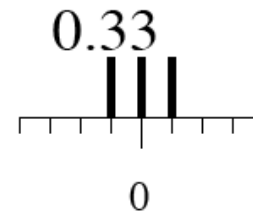
Sharpening Filter



Original



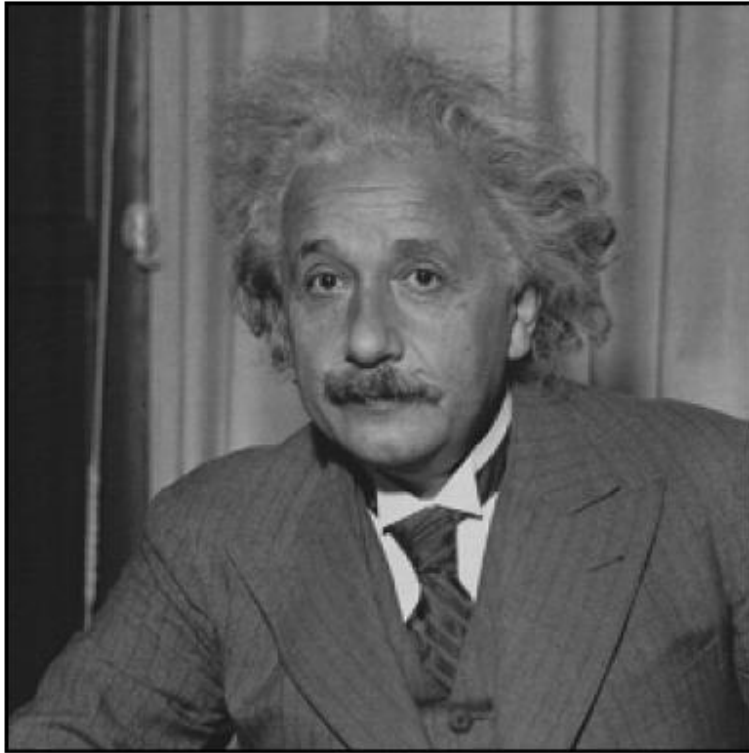
—



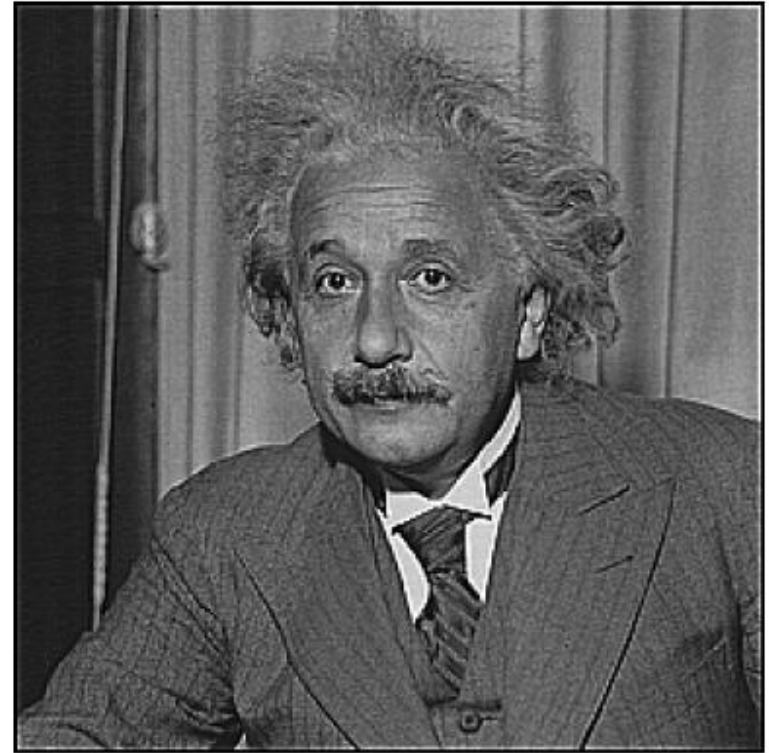
Sharpening filter

- Accentuates differences with local average

Sharpening Filter



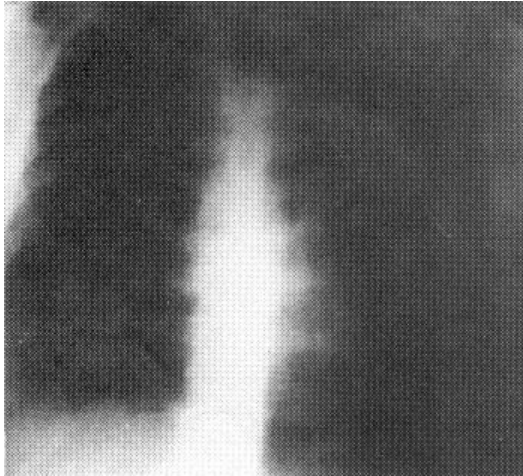
before



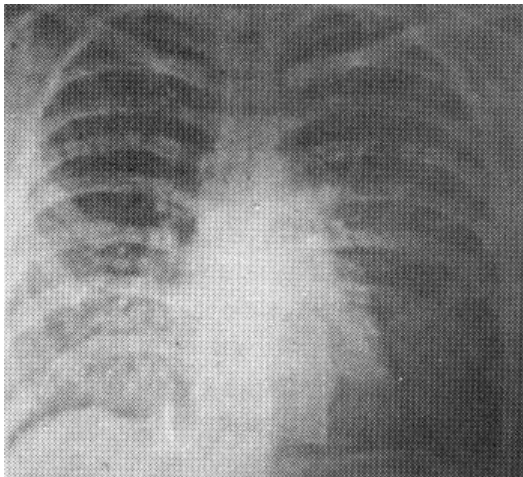
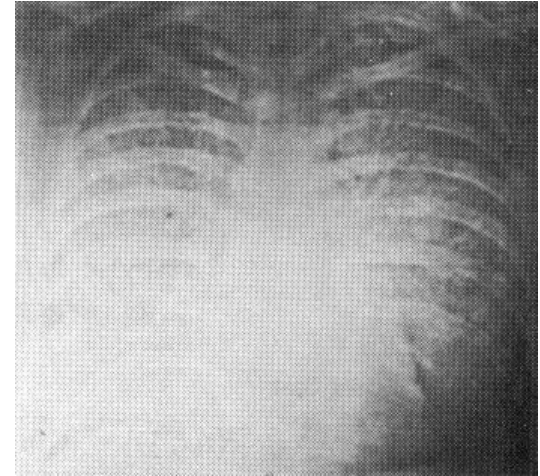
after

Application: High Frequency Emphasis

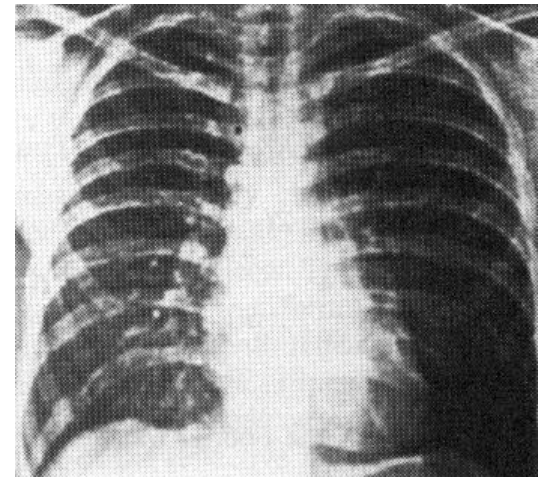
Original



High pass Filter



High Frequency
Emphasis



High Frequency Emphasis
+
Histogram Equalization

Topics of This Lecture

- Linear filters
 - What are they? How are they applied?
 - Application: smoothing
 - Gaussian filter
 - What does it *mean* to filter an image?
- Nonlinear Filters
 - Median filter
- Multi-Scale representations
 - How to properly rescale an image?
- Image derivatives
 - How to compute gradients robustly?

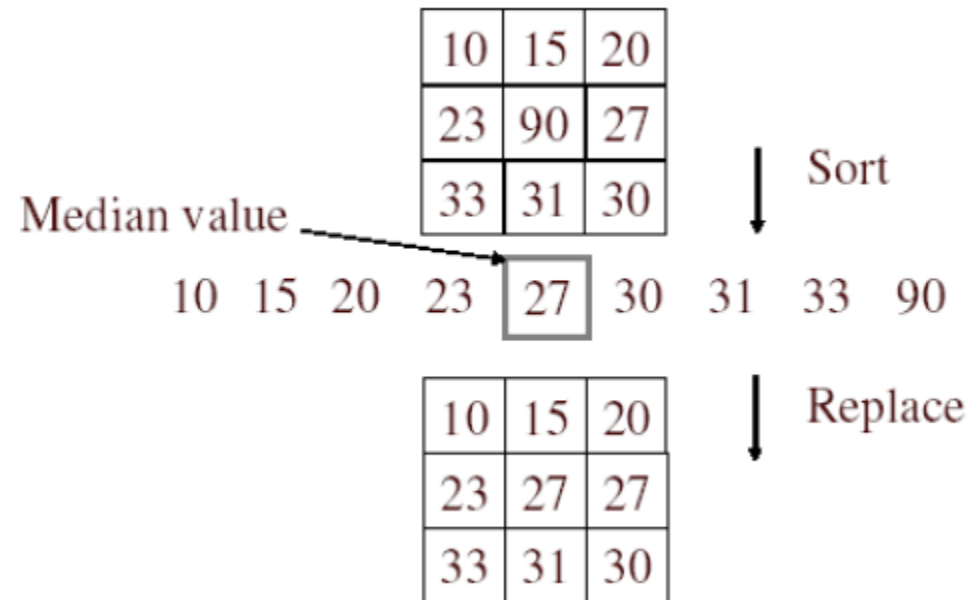
Non-Linear Filters: Median Filter

- Basic idea

- Replace each pixel by the median of its neighbors.

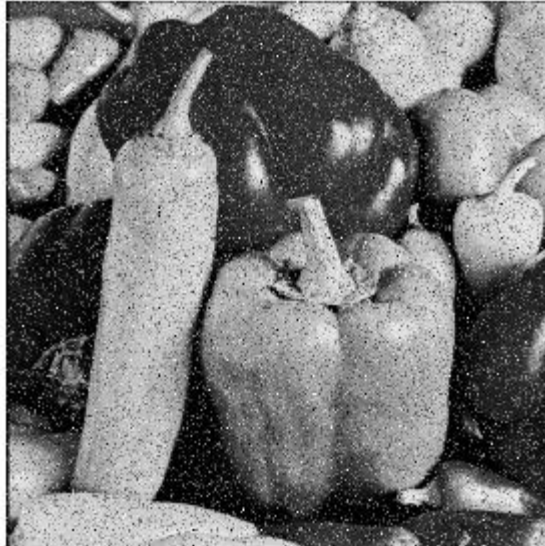
- Properties

- Doesn't introduce new pixel values
- Removes spikes: good for impulse, salt & pepper noise
- Linear?

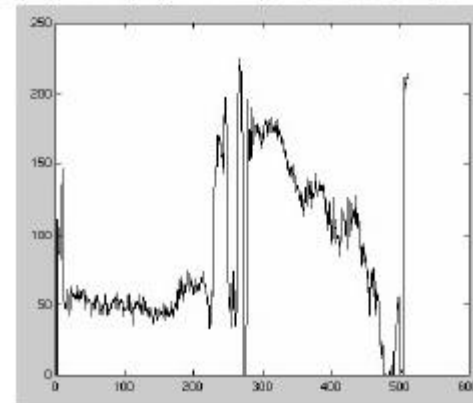
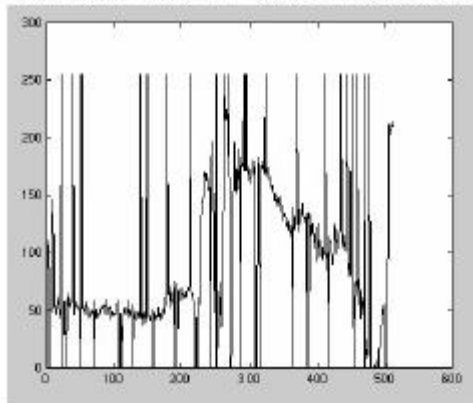


Median Filter

Salt and
pepper
noise



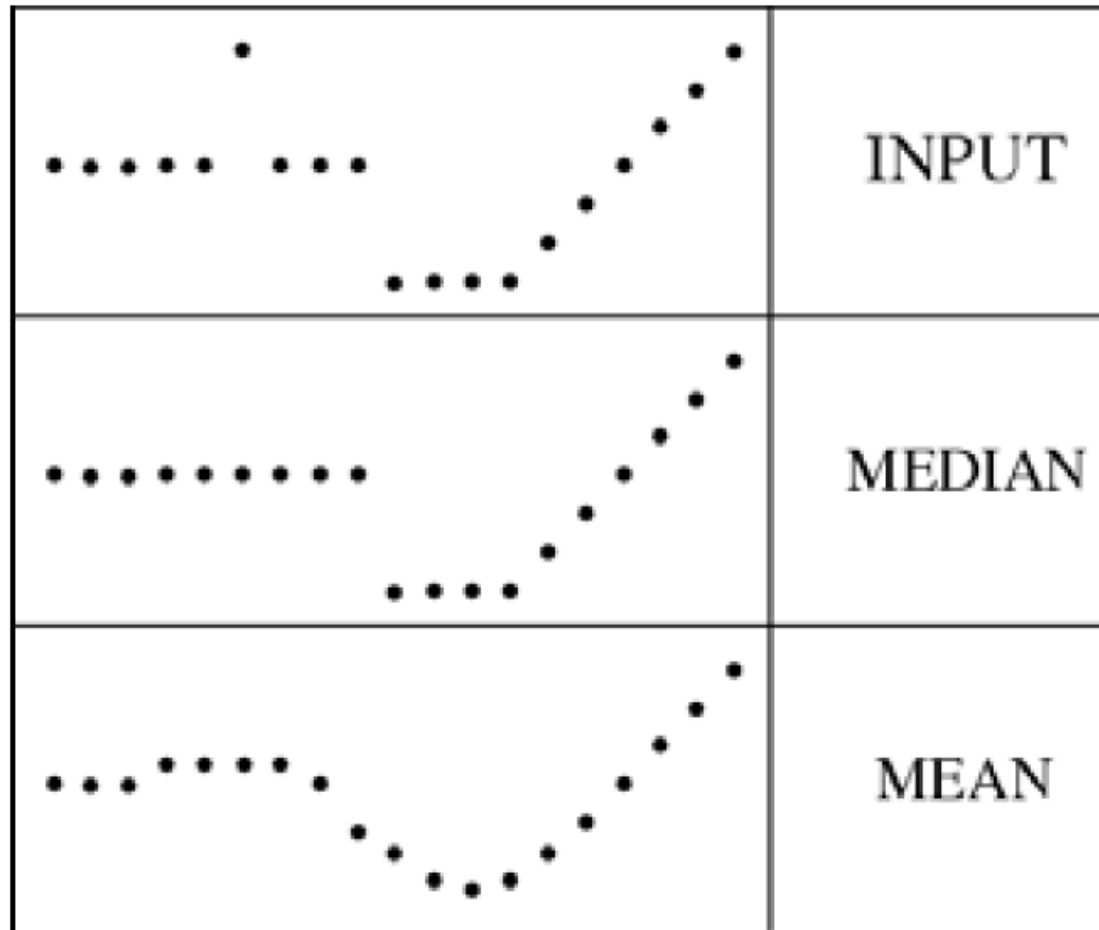
Median
filtered



Plots of a row of the image

Median Filter

- The Median filter is **edge preserving**.



Median vs. Gaussian Filtering

3x3

5x5

7x7

Gaussian



Median



Topics of This Lecture

- Linear filters
 - What are they? How are they applied?
 - Application: smoothing
 - Gaussian filter
 - What does it *mean* to filter an image?
- Nonlinear Filters
 - Median filter
- Multi-Scale representations
 - How to properly rescale an image?
- Filters as templates
 - Correlation as template matching



To Be Continued...

...in the next lecture.

References and Further Reading

- Background information on linear filters and their connection with the Fourier transform can be found in Chapter 3 of the Szeliski book.

R. Szeliski
Computer Vision – Algorithms and Applications
Springer, 2010
([draft version available online](#))

